

Lecture: 27- How to train your LLM

LLM Pretraining loop Schematic

Output $\equiv$ targets

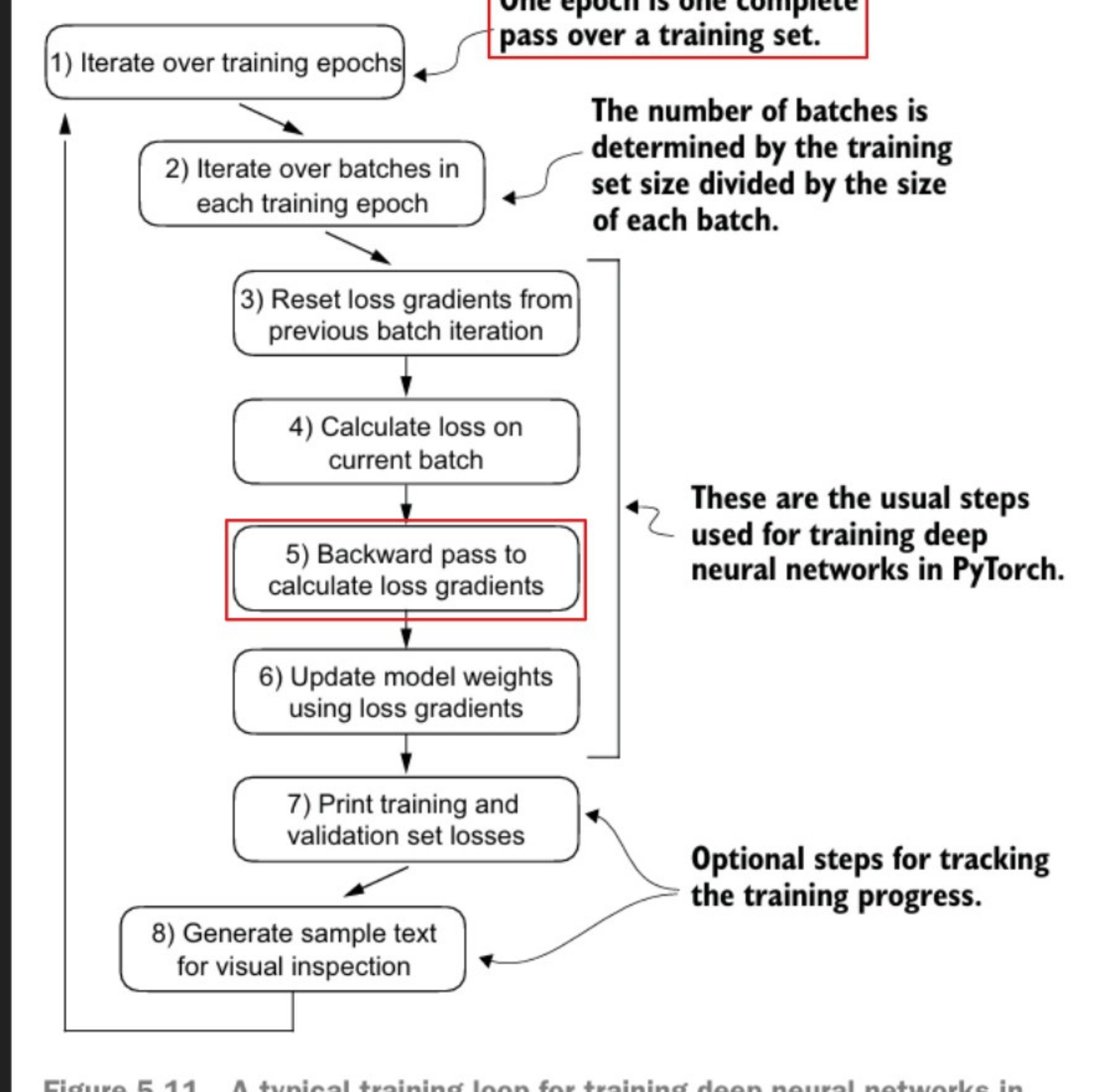
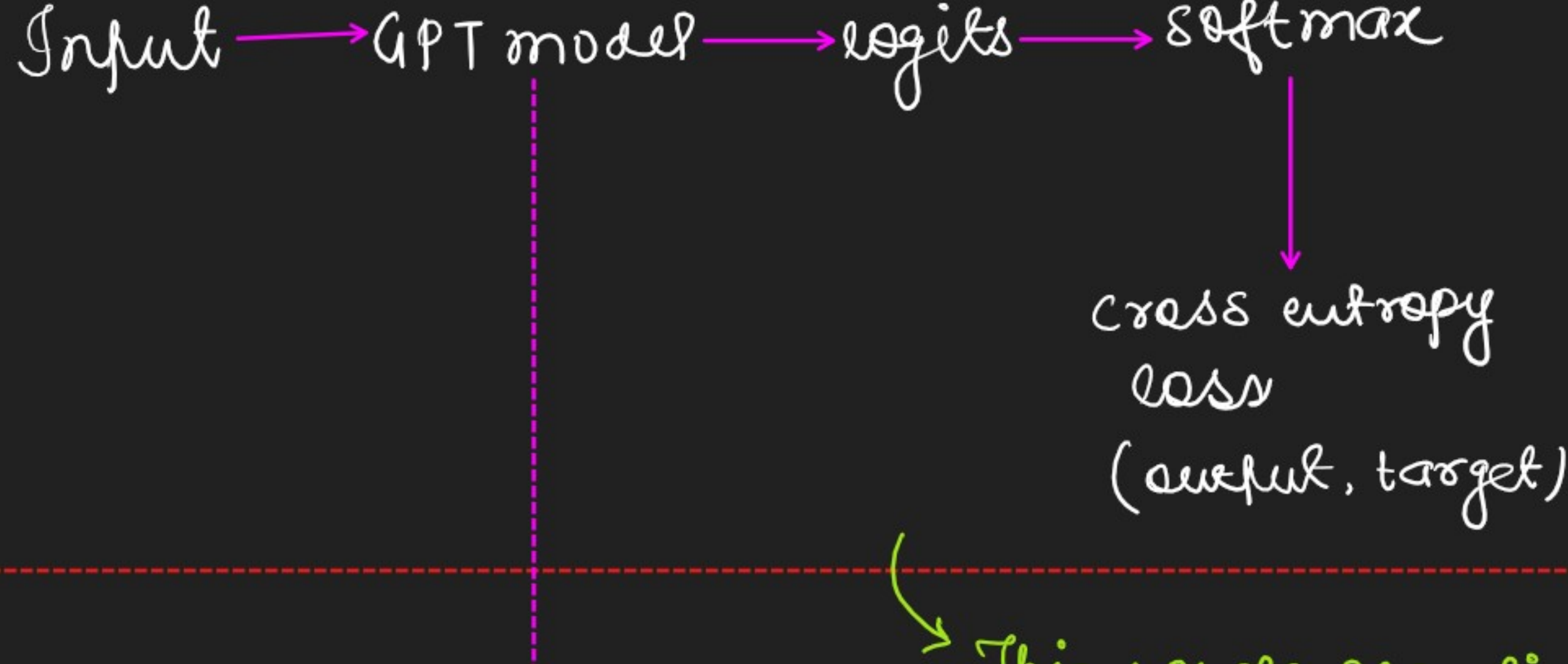


Figure 5.11 A typical training loop for training deep neural networks in PyTorch consists of numerous steps, iterating over the batches in the training set for several epochs. In each loop, we calculate the loss for each training set batch to determine loss gradients, which we use to update the model weights so that the training set loss is minimized.

Main step: Find loss gradient using `loss.backward()`



This whole operation is differential.

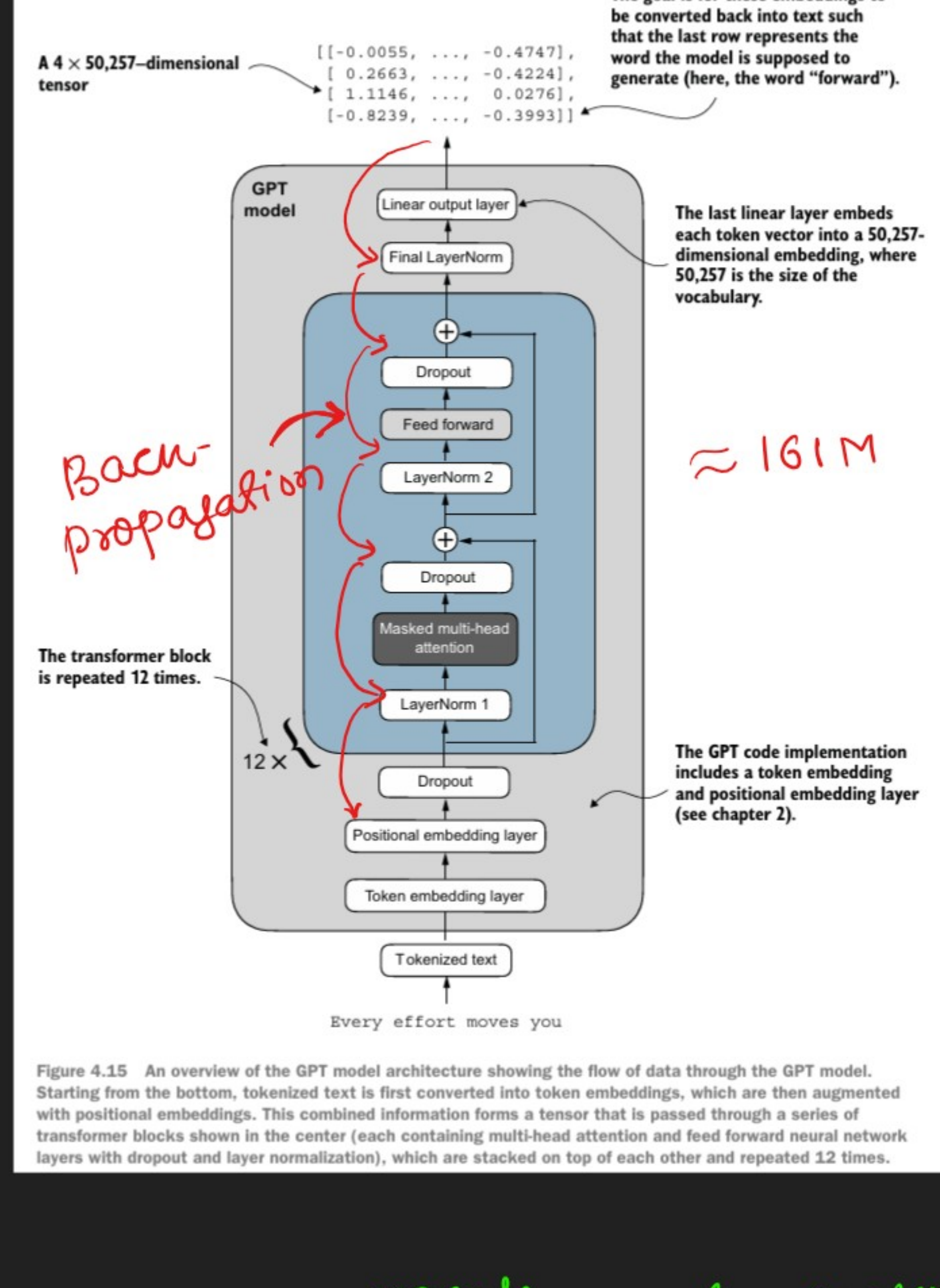


Figure 4.15 An overview of the GPT model architecture showing the flow of data through the GPT model. Starting from the bottom, tokenized text is first converted into token embeddings, which are then augmented with positional embeddings. This combined information forms a tensor that is passed through a series of transformer blocks shown in the center (each containing multi-head attention and feed forward neural network layers with dropout and layer normalization), which are stacked on top of each other and repeated 12 times.

$$\text{* Embedding parameter} : \underbrace{50257 \times 768}_{\text{token}} + \underbrace{1024 \times 768}_{\text{positional}} \approx 38.4M$$

$$\begin{aligned} \text{* Transformer block parameter} &= \text{* multi head attention} \\ &\rightarrow \text{Query, key, value weight} \\ &= 3 \times 768 \times 768 = 1.77M \\ &\rightarrow \text{Output head} \\ &= 768 \times 768 = 0.59M \\ &\rightarrow \text{Total: } 2.36M \end{aligned}$$

* Feed forward neural network

$$768 * (4 * 768) + 768 (4 * 768) = 4.72M$$

Total for 12 transformer block:

$$= 12 * (2.36 + 4.72)$$

$$\approx 85.2M$$

$$\text{* Final layer (softmax output): } 768 \times 50257 : 38.4M$$

$$\text{Total Parameters to be optimised} = 38.4 + 85.2 + 38.4 \approx 162M$$

$$\text{GPT-2 parameter} = 124M$$

weight tying

For each of this parameters, we need to perform gradient descent updates:

$$w = w - \alpha \frac{\partial L}{\partial w}$$

update model weight in each iteration.

we calculate $\frac{\partial L}{\partial w}$ in back-ward pass.

loss function.

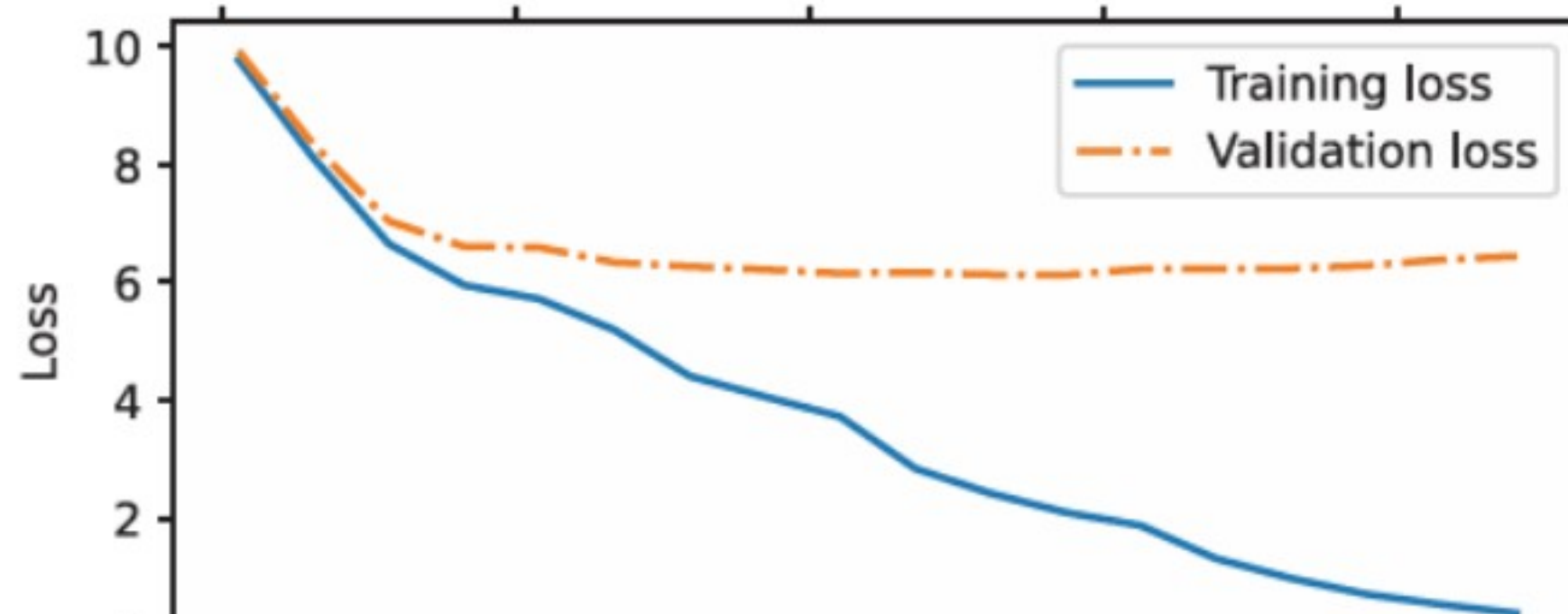
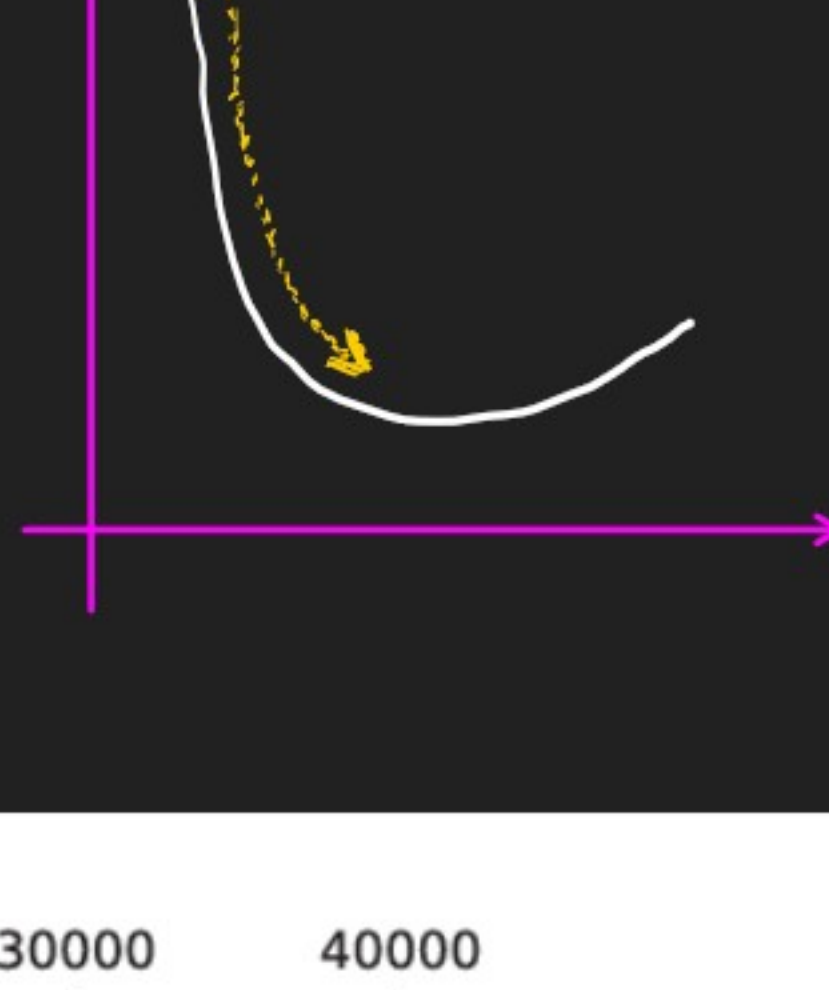


Figure 5.12 At the beginning of the training, both the training and validation set losses sharply decrease, which is a sign that the model is learning. However, the training set loss continues to decrease past the second epoch, whereas the validation loss stagnates. This is a sign that the model is still learning, but it's overfitting to the training set past epoch 2.